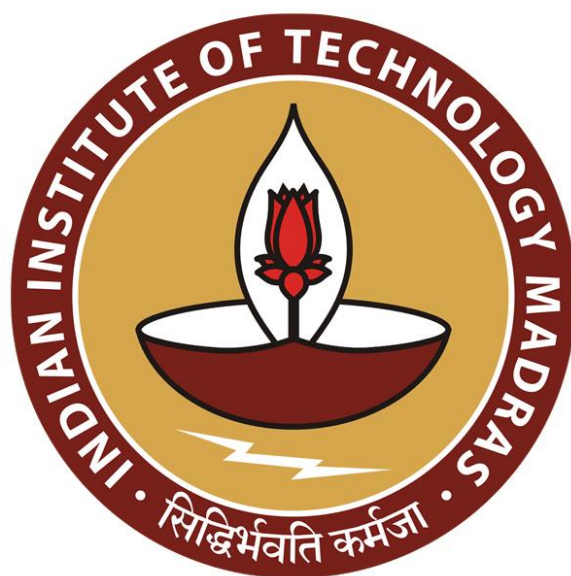


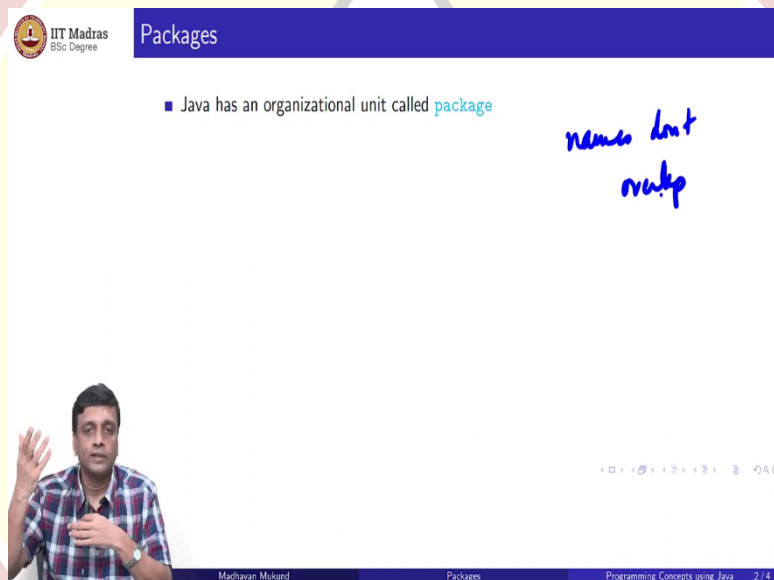


IIT Madras
ONLINE DEGREE

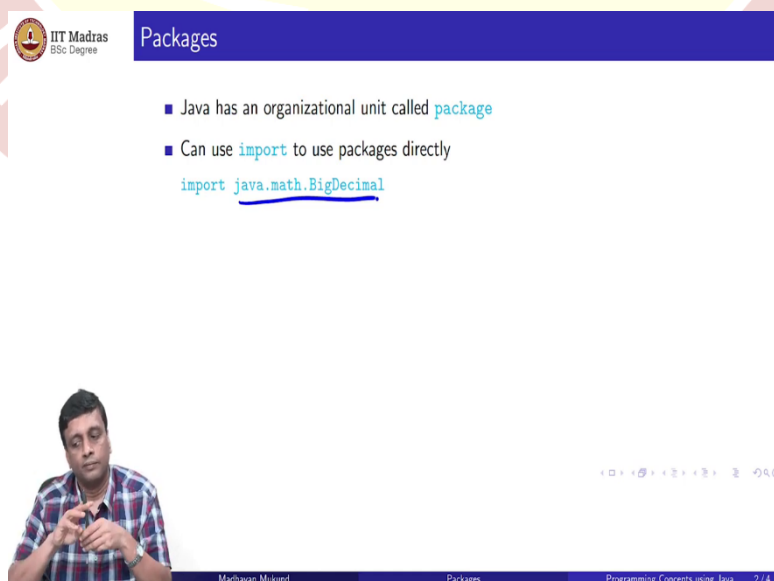
Programming Concepts Using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Packages

So, we have been looking at errors and exceptions. So, let us digress a little bit and look at something quite different. Because we will need it a little bit. There is something called a package. So, what we saw is that Java organises all the data and the programme using this hierarchy of classes. And so, we have one public class per file, and we have private classes, you may see it inside those files, and so on.

(Refer Slide Time: 00:39)



The slide is titled "Packages" and features the IIT Madras logo in the top left corner. The main content area contains a bullet point: "Java has an organizational unit called `package`". To the right of this text, there is handwritten blue text that reads "names don't repeat". In the bottom left corner, there is a small video inset showing Professor Madhavan Mukund speaking. The bottom of the slide has a navigation bar with the text "Madhavan Mukund", "Packages", "Programming Concepts using Java", and "2 / 4".



The slide is titled "Packages" and features the IIT Madras logo in the top left corner. The main content area contains two bullet points: "Java has an organizational unit called `package`" and "Can use `import` to use packages directly". Below the second bullet point, there is a code snippet: `import java.math.BigDecimal`, which is underlined in blue. In the bottom left corner, there is a small video inset showing Professor Madhavan Mukund speaking. The bottom of the slide has a navigation bar with the text "Madhavan Mukund", "Packages", "Programming Concepts using Java", and "2 / 4".

- Java has an organizational unit called `package`
- Can use `import` to use packages directly

```
import java.math.BigDecimal
```

- All classes in `.../java/math`

```
import java.math.*
```

specific pkg in java.math



Madhavan Mukund

Packages

Programming Concepts using Java 2/4

- Java has an organizational unit called `package`
- Can use `import` to use packages directly

```
import java.math.BigDecimal
```

- All classes in `.../java/math`

```
import java.math.*
```

- Note that `*` is not recursive. Cannot write `import java.*`



Madhavan Mukund

Packages

Programming Concepts using Java 2/4

But Java also has an organisational unit called a package. And one reason for this is that you might of course have different classes, which written in different contexts which have the same name. So, we need to be able to distinguish between a class which has a name in this context of class, which has a name and that context. So, this package is really a kind of way of organising your names so that they do not overlap.

So, names do not overlap. So, if I developed some code, then I can be sure that the names that I am using can be disambiguated from any names that you might use in your code, by putting them into a package and saying that in my package, I have this class, it is in your package, you have a different class. So, with the class and the package name these two things can be distinguished.

So, if I want to now import functions, which are defined in other packages, I can just give the usual package name. So, what this does is these imports all the functions which are in the package `java dot math dot BigDecimal`. So, this is as we will see Java's typical terminology for packages, which is separated by dots, which indicates a hierarchy.

So, Java is in this case, the kind of biggest is the kind of parent class of pack it is like a hierarchy of packages. So, Java is bigger than `Java dot map` is smaller and `BigDecimal` is a particular package inside `java dot map`. Now, if I want all the classes in `java dot math`, then I can import `java`. Now, technically speaking, this hierarchy in the name is also reflected in a hierarchy and how these files are stored on the disk.

There is no direct reason why this should be the case, but this is how Java is typically organised. So, if you have a Java system and you want to find out the files corresponding to `big decimal`, you would look at the you will locate the package directory for Java and underneath that there will be a subdirectory or sub folder called `math` and inside that there will be something called `decimal`.

So, if you want everything under `math` all the packages under `math`, you can say `java dot math dot star`, but these are all specific packages. So, there is no next level after `java dot math` there is not `Java dot math dot whatever`, `integer math` or `Java dot math` or `double math`, which has a further level of packages. All the `math` packages are in this one thing under one level called `Java dot math`.

So, when I say `java dot math dot star`, it says import all the individual packages at this level. But I cannot say import everything that Java has, I cannot say `java dot star` which will say `java dot math dot something java dot lang dot something`. So, Java has a lot of these packages organised in the hierarchy. So, under `math`, you have all the `math` packages under `lang`, who have some very language specific package. So, I cannot say `java dot star` and get them. So, I can only use the star at the bottom level of this package hierarchy. So, that is one important thing.

(Refer Slide Time: 03:37)



Creating and naming packages

- Can create our own hierarchy of packages
- Naming convention is similar to Internet domain name, but in reverse
 - Internet domain: `onlinedegree.iitm.ac.in`
 - Package name: `in.ac.iitm.onlinedegree`

The screenshot shows a video lecture interface. At the top, a presenter is visible. Below him is a slide titled "Creating and naming packages" with the IIT Madras BSc Degree logo. The slide content is identical to the one above, listing the naming convention for packages. The slide is part of a presentation titled "Programming Concepts using Java" and is slide 3/4. The presenter's name "Machavan Mukund" is visible at the bottom of the slide. The slide content is as follows:

Creating and naming packages

- Can create our own hierarchy of packages
- Naming convention is similar to Internet domain name, but in reverse
 - Internet domain: `onlinedegree.iitm.ac.in`
 - Package name: `in.ac.iitm.onlinedegree`

Creating and naming packages

- Can create our own hierarchy of packages
- Naming convention is similar to Internet domain name, but in reverse
 - Internet domain: `onlinedegree.iitm.ac.in`
 - Package name: `in.ac.iitm.onlinedegree`



Machavan Mukund

Packages

Programming Concepts using Java

3 / 4

Creating and naming packages

- Can create our own hierarchy of packages
- Naming convention is similar to Internet domain name, but in reverse
 - Internet domain: `onlinedegree.iitm.ac.in`
 - Package name: `in.ac.iitm.onlinedegree`
- Add a package header to include a class in a package

```
package in.ac.iitm.onlinedegree;  
  
public class Employee { ... }
```



Machavan Mukund

Packages

Programming Concepts using Java

3 / 4

Creating and naming packages

- Can create our own hierarchy of packages
 - Naming convention is similar to Internet domain name, but in reverse
 - Internet domain: `onlinedegree.iitm.ac.in`
 - Package name: `in.ac.iitm.onlinedegree`
 - Add a package header to include a class in a package
- ```
package in.ac.iitm.onlinedegree;

public class Employee { ... }
```
- By default, all classes in a directory belong to same anonymous package



Machavan Mukund

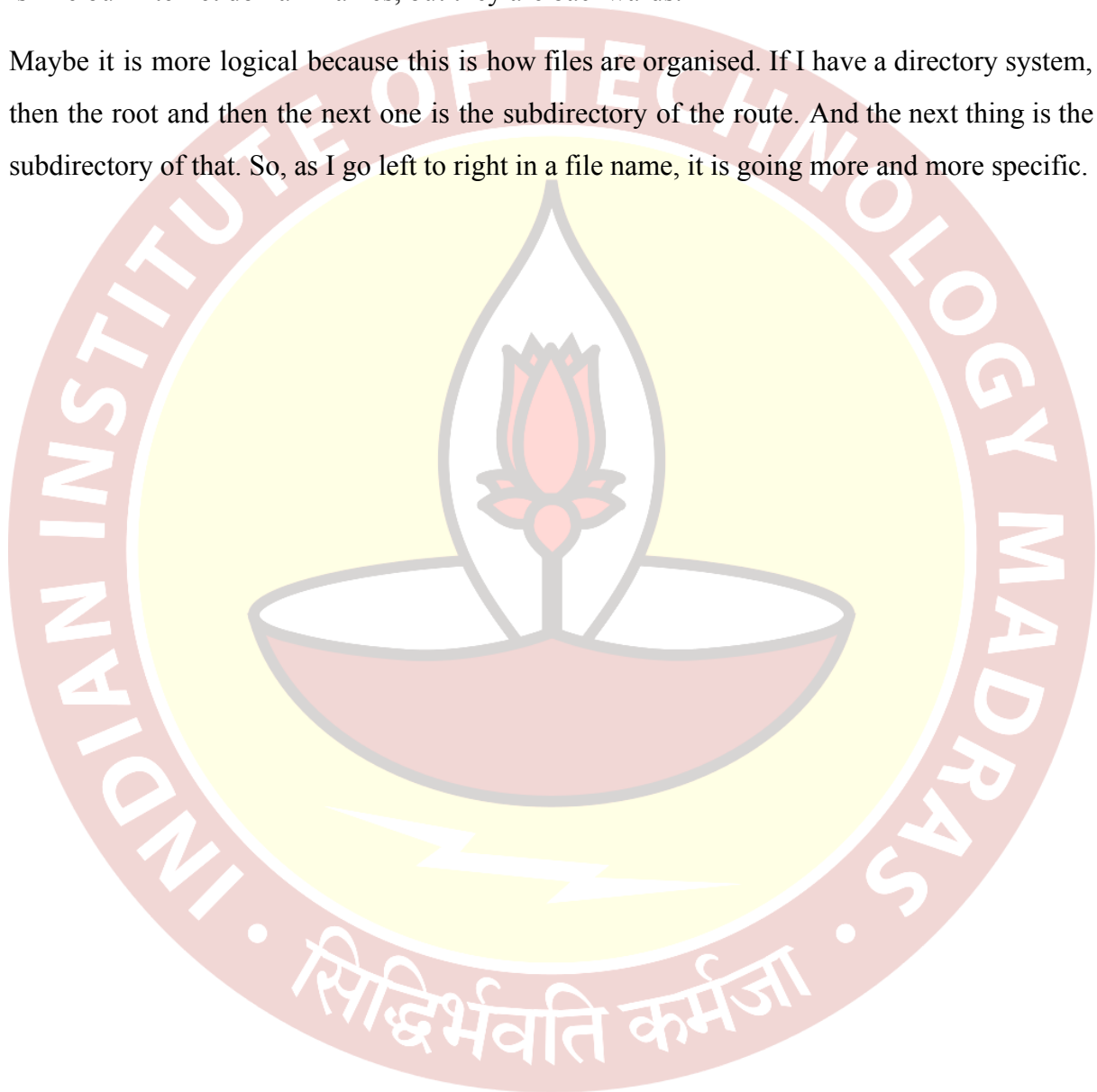
Packages

Programming Concepts using Java

3 / 4

So, this is as far as the built-in packages. But as I said, the reason is to disambiguate. So, when I write a package or rewrite a package, it we should be able to tell each other classes apart. So, I should not be forced to use different names just because somebody else's use the same name. So, we can create our own hierarchy of packages. And we use normally in Java naming convention using dots I said, and which is kind of hierarchical, except the hierarchy is like our internet domain names, but they are backwards.

Maybe it is more logical because this is how files are organised. If I have a directory system, then the root and then the next one is the subdirectory of the route. And the next thing is the subdirectory of that. So, as I go left to right in a file name, it is going more and more specific.





Whereas internet actually goes the other way right so dot in indicates a domain that is in India and dot ac dot in indicates an academic institution inside India and then I iitm dot ac dot in indicates all the domains which belong to IIT Madras and so on. So, then the Java name for this will be the other way around. So, I start with in then I go to ac then I go to iitm and online degree. So, this is just a naming convention. And this we will often see in Java.

So, when I now create a class and I want to put it in a specific package, then I use the package name as an extra header line inside my class definition. So, this is my actual normal class description, I just add on top at the beginning of the class file, the class dot java file, I indicate the package name by saying package followed by the name of the package and a semicolon.

So, this is an indicator to Java to put this particular class in that package. And if as we have been doing, there is no explicit package name, given then which package do they belong to? So, it turns out that everything that is sitting in the same directory or folder if you prefer that word, belongs to the same package, which is anonymous. So, anything which is not explicitly part of a package through a name given in the class file, is actually belong belongs to the same package, which is a kind of anonymous package.

So, everything that so if you write, that is why if you write a bunch of classes, we have no difficulty finding them, we do not import them, because they are all sitting inside the same directory. So, they are implicitly available to us without saying import this directory, that class and import this directory, that class.

(Refer Slide Time: 06:02)



## More about visibility

- We have seen modifiers `public` and `private`
- If we omit these, the default visibility is public *within* the package
  - This applies to both methods and variables



Madhavan Mukund

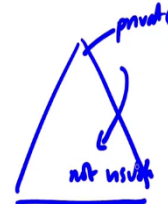
Packages

Programming Concepts using Java

4 / 4

## More about visibility

- We have seen modifiers `public` and `private`
- If we omit these, the default visibility is public *within* the package
  - This applies to both methods and variables
- Can also restrict visibility with respect to inheritance hierarchy



Madhavan Mukund

Packages

Programming Concepts using Java

4 / 4

## More about visibility

- We have seen modifiers `public` and `private`
- If we omit these, the default visibility is public *within* the package
  - This applies to both methods and variables
- Can also restrict visibility with respect to inheritance hierarchy
  - `protected` means visible within subtree, so all subclasses
  - Normally, a subclass cannot expand visibility of a function

public  
|  
protected  
|  
private



Madhavan Mukund

Packages

Programming Concepts using Java

4 / 4

- We have seen modifiers `public` and `private`
- If we omit these, the default visibility is public `within` the package
  - This applies to both methods and variables
- Can also restrict visibility with respect to inheritance hierarchy
  - `protected` means visible within subtree, so all subclasses
  - Normally, a subclass cannot expand visibility of a function
  - However, `protected` can be made `public`



Machavan Mukund

Packages

Programming Concepts using Java

4 / 4

So, this brings us to another topic. So, we have seen these modifiers `public` and `private`, but in some examples, and in various books, you will often see that people omit these modifiers. So, this neither `public` nor `private`, so what could it possibly mean? So, in Java, what this means is something which is visible within the package.

So, that class is if anything is defined to be just a class or just a method, or just an instance variable without saying `public` or `private`, it means that it is `public` within the class within the package. So, the every, remember that every class either belongs to an explicitly named package, which is identified in the top of the Java file, or it belongs implicitly to this anonymous class, which is shared by all the other classes in the same directory in which it sits.

So, this default visibility of being `public` applies to methods and variables. Now, one could argue that this is natural with methods because you do want to use methods as a way of accessing variables, rather than the variables themselves. But unfortunately, Java does not have a mechanism to distinguish between this. So, if you leave, I mean, one way could have been to say that you could only leave out `public` and `private` for method, but you are forced to put in `public` and `private` variables.

But Java has chosen this option of allowing you to omit it for both and so, you can either omit it for variables or for methods or both and wherever you omit it, it becomes `public` within a package that means that if you actually write this, these class definitions and you do not put a

private then other functions and other classes in the same directory will be able to update the values even though it is not explicitly public.

So, although it is not directly related to packages, let us talk about one more thing that is possible, which is that now, you have, we have talked about the fact that we have a class hierarchy and if something is private here, it is not visible here. So, we said for instance, that if a manager is a special type of employee and employee has some private information about say salaries, the manager cannot directly access that private field and modify it, it has to do it through the public interface that the employee class provides.

Now, there may be situations where you trust the classes that extend to do it, but not all classes. So, you want something which is public within the hierarchy, but private outside the hierarchy. So, this is something called protected. So, protected is an attribute which says that it is public to any class it is public to this class of course, because everything is public within a class, but to any subclass.

So, anything which explicitly extends this class can see this as public anything which does not extend this class, it is private. So, when you go down this thing, then it turns out that protected. So, we have kind of if you think of the strictness, so, within a hierarchy private is the most top, if I replace something to private than nothing below me can see it only I can see it.

If I declare some new public not only everything below me with everything are all over any class can see it and protected means it can only be seen like this. So, protected is limited public it is public within the instance hierarchy. Now, in the normal rule of Java is that when you extend something or when you redefine something or overwrite something, you can reduce its visibility, you cannot expand it.

So, you cannot change in general visibility by making something that was private public. But it turns out that you are allowed to make something that is protected in a higher class, public in your class. And this will be used later on when we talk about cloning, which is a way of copying one object to another object. So, this is a small digression which is not directly about packages, but it is connected in a sense because packages give us the visibility through this lack of a modifier, but we have now this new modifier called protected which tells us about visibility within instance hierarchy of a class.